

kid_buddy — 会讲故事的儿童 AI 伙伴

2026 首届 openvela AI 硬件开发者大赛 · 参赛作品介绍

作品名称 kid_buddy

所属赛道 AI 硬件产品创新

队伍编号 086

硬件平台 Gemini-S1 (全志 R528 / sun8iw20, 128M NAND, 单麦 + 喇叭 + LCD)

专属仓 https://github.com/open-vela/contest2026_086_ditiekouyegoulianmeng

应用版本 v2.56

一、这是什么

一块巴掌大的板子，加一个能听会说的儿童玩伴。

孩子对着它喊一声「你好，openvela」，它醒过来。之后可以随便聊天问问题，也可以**挑一个角色进入一段冒险故事**——老师、故事家、科学家、朋友，每个角色的说话方式和故事线都不一样。故事是**有分支**的：孩子选「推开木门」还是「绕到屋后」，后面走的是两条路。

跟别的语音助手相比，它有三个地方是刻意做的：

故事和闲聊不混在一起。孩子中途问一句「恐龙为什么会灭绝」，不会把剧情带跑偏。两个会话在系统里是分开存的，回来接着讲还在原来那条线上。

孩子走开了会自己接上。讲了一半人跑了，过一会儿它会主动问：「上次的冒险讲到一半啦，还要继续

吗？」

回答完还给一次机会。 回复完的十几秒内，孩子直接说「第二个」或者「继续」就行，不用再喊一遍唤醒词。小孩说不出「请重复上一句」这种话，这个窗口是替他们省事的。

另外，孩子说「我懂啦」或者「这个我不太明白」的时候，会通过 MQTT 推一条确认消息到家长端——不是监控，是让家长知道今天聊到哪儿了。

二、怎么跑起来

方式一：直接烧预编译固件（推荐）

```
image/kid_buddy_gemini-s1_uart0_128Mnand_v2.56.img
```

用全志 PhoenixSuit / LiveSuit 按常规流程烧进 NAND。上电后启动脚本会自动拉起图形界面和语音后台，不需要再编任何东西。

烧完屏幕上右上角会显示版本号（v2.56），可以用来确认新镜像确实生效了。

方式二：从源码编

本仓自带完整的板级配置（board/nsh_minidisplay/），不需要另外改 vendor 树：

```
repo init -u https://github.com/open-vela/contest2026_086_ditiekouyegoulianmeng \
  -b dev-ai-contest-2026 -m contest2026_086_ditiekouyegoulianmeng.xml
repo sync -c -j8

# 在工作区根目录
./build.sh contest2026_086_ditiekouyegoulianmeng/board/nsh_minidisplay
```

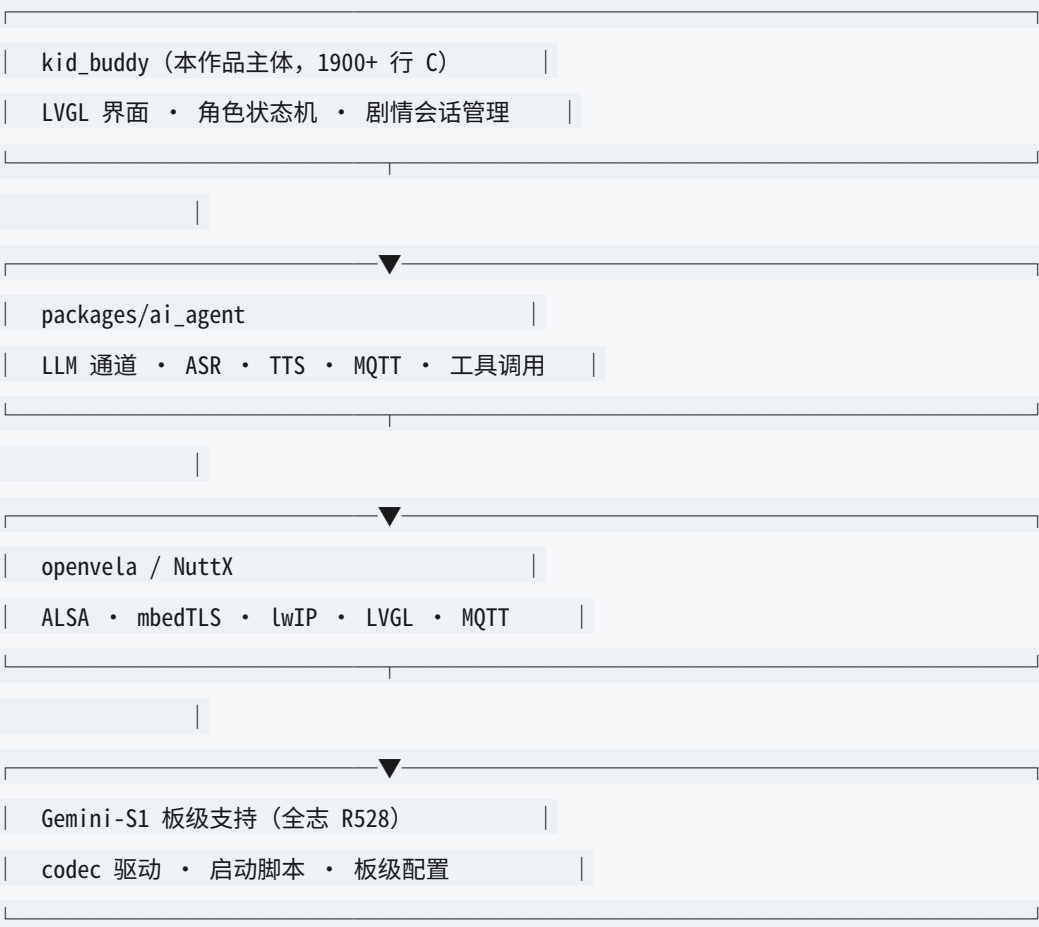
编译完成后打包：

```
cd vendor/allwinnertech/lichee
./tools/scripts/pack_img.sh -c sun8iw20p1 -p rtos -b r528s3-gemini-s1 -o nuttx \
-d uart0 -s none -m normal -w none -v none -i none -t "$PWD" \
-f r528s3/gemini-s1_nand -g r528s3/gemini-s1_nand
```

注意：改过 packages/ai_agent 的头文件之后，必须 make -C nuttx distclean 再编。这套构建不跟踪头文件依赖，留着旧的目标文件会出现 结构体大小对不上的问题。make clean 不够——它不清 registry/ 和 builtin_list.co。

三、技术方案

3.1 整体结构



3.2 语音链路

采集：本地能量阈值 VAD 掐出一句话（20ms 一帧，起止阈值带迟滞），只在没有回复在播 的时候才开麦克风。

识别：音频送 MiMo ASR 转写。

唤醒：板子上没有本地 KWS 引擎，唤醒是在**转写结果上做归一化匹配**——大小写折叠、空格标点当分隔符、并认几个音译写法。这样「你好，openvela」「你好 openvela」「Hello, OpenVela」都能唤醒。

合成：MiMo TTS，一次回复只开关一次音频设备，按句流水线预取。

3.3 剧情会话隔离

这是这个作品里最能体现「不是套壳」的一块。

自由聊天和冒险故事如果共用一份对话历史，剧情状态会被闲聊污染（反过来也一样：家长问一句天气，故事里的角色会突然开始聊天气）。

做法是：进入故事模式时，这一路的请求带一个**独立的会话标识**，`ai_agent` 的会话管理器就把它的历史单独存一份。进出故事模式由语音触发（「我要听故事」/「不玩了」），状态落盘，中途断电再开机还能接着上次的剧情。

3.4 角色与技能

四个角色各是一份 Markdown 文件（`app/kid_buddy/skills/role-*.md`），里面是人设和剧情大纲。运行时载入并注入提示词，所以**换角色、改人设不用重新编译**——加一份文件就多一个角色。

四、做这个作品踩过的坑（也是它值钱的地方）

这部分写在作品介绍里，是因为它代表了真实的工程量。每一条都是先误判、再定位、最后改到驱动层或协议层才解决的。

现象	真正的原因	层次
只有第一句回复有声音，追 codec 每次打开都无条件重配 DAC 时钟，把时钟打毛；且 dapm_state 没重置，模问全哑	拟通路不再重跑	内核驱
播放时麦克风报 request dma chan failed	单麦板子上采集与播放共享同一条 codec DMA，忙标志置晚了	驱动 +
老是自己唤醒自己	单麦板子用了 Kconfig 默认的采集增益 6，底噪被放大 36 倍顶穿 VAD	配置
TTS 时有时无	云端端点是个负载均衡，个别后端 IP 在某段时间握手必失败	网络协
每句话卡二十秒	TLS 客户端对 keep-alive 的 chunked 响应一直读到服务端关连接，而 keep-alive 就是不会关	协议实
提醒到点不响	单线程的消息分发被一个卡住的 MQTT 连接阻塞，整条队列出不去	并发
多轮对话返回上一次的答案	回复缓存的键用了被截断的当前消息，前缀相同时撞键	状态管
结构体字段读出 240 这样的垃圾值	构建系统不跟踪头文件依赖，改过头文件后链接的是旧目标文件	构建系
改过启动脚本、重编也报成 构建系统 只把 etc/init.d/rcS 列为依赖，而它全部内容是 #include "rcS.nsh"，rcS.nsh 压根不在依赖表里，改它永远不触发重生成功，行为却没变		构建系
烧录后白屏，只有右下角一板级配置里 CONFIG_LCD_FRAMEBUFFER=y 会让驱动的 #if/#elif 走 framebuffer 分支，lcddev_register() 成了死代码，/dev/lcd0 压根没注册出来		板级酉
配好网络和 APIKEY 之后毫无反应	板子没有 RTC，系统时钟停在 1970，HTTPS 证书的 notBefore 校验全部失败	系统 / 协议
这些改动里，与 openvela 公共仓有关的部分都按大赛规则单独提了 PR，没有直接改本地副本。		

五、AI 开发流程

全程使用 Claude Code。从 7 月 25 日到 9 月 12 日，25 个有产出的开发日，完整对话 记录在 logs/ 目录里。

AI 在这个项目里主要不是「写代码」，而是在信息不足的时候帮忙缩小范围：

- 上面那张表里的「真正的原因」，大部分是先由 AI 根据日志和代码提出假设、再去验证的。比如「TTS 时有时无」一开始怀疑是证书算法不被支持，去查那个 IP 才发现证书是健康的，真正的问题是负载均衡的坏窗口。
- 构建系统的坑（陈旧目标文件、配置不同步）也是 AI 从「代码明明改了但行为没变」这个现象反推出来的。

沉淀的技能。我们没有重写官方技能集已经覆盖的东西（编译、驱动、音频质量分析、提 PR 流程），而是把官方没覆盖、我们踩出来的部分写成了 4 个 Skill，放在本仓 .claude/skills/：

Skill	内容
openvela-build-traps	陈旧目标文件、clean 与 distclean 的区别、build.sh 会覆盖板级配置、第三方 app 补丁失效
ai-agent-board-bringup	ai_agent 的配置矩阵、按 IP 回退的 TLS 重试、chunked 响应读不完、MQTT 双线程互踢与阻塞
voice-loop-tuning	单麦增益、DMA 争用、codec 二次打开无声、唤醒词归一化匹配
openvela-contest-submit	ruleset 与 branch protection 的区别、CLA 邮箱、只能 rebase 合并导致 SHA 重写

每个 Skill 的 description 里都写了触发条件（Use when: ...），Claude Code 会在遇到 对应症状时自动选用。

六、已知限制

写在前面，不藏：

1. **语音链路全在云端。** LLM、ASR、TTS 都走网络，断网时只剩图形界面可用。
 2. **唤醒不是本地声学唤醒。** 板子上没有 KWS 引擎，实际是 VAD + 云端 ASR 转写后做字符串匹配，所以环境吵的时候得离近一点说。唤醒词的匹配写法做过离线单元测试（含正例和不该唤醒的负例），**但没有针对真实 ASR 输出做过大规模校准**——日志里会打印原始转写，遇到没覆盖的写法加一条即可。
 3. **上游 PR 尚未合入。** 作品对 `packages_ai_agent` 和 `vendor_allwinnertech` 的改动已按规则提了 PR，但组委会的评审有 CODEOWNERS 闸门，参赛者无法自行合并。因此 **本仓的预编译固件是实际交付方式**——镜像里已经包含了这些改动，烧录后功能是完整的。
 4. **预编译固件的验证程度。** 当前交付的 v2.56 修掉了两个**从真机串口日志里定位出来**的问题（`/dev/lcd0` 未注册导致白屏、系统时钟停在 1970 导致 HTTPS 全部失败，见第四节）。时钟这一条的具体修法是让 `kid_buddy` 启动时自己调 `clock_settime`（NuttX 的 `date -s` 底下就是这个调用），并且只在时钟明显没设过的时候才写，**这样从源码编出来的固件也覆盖得到**——原先这一步写在 vendor 的 `rcS.nsh` 里，只有本仓这份预编译镜像才带。**白屏一项已在真机上复烧确认修复**（v2.55 上板：图形界面正常显示、版本号对得上）。镜像还做过构造校验：`nsh.fex` 与 `vela.bin` 同为 5828112 字节且逐字节一致；版本号 v2.56、内建程序名、以及新增的时钟种子代码在镜像里都查得到；**内嵌的那份启动脚本里已经不再有 `date -s`**；直接搜镜像二进制，新唤醒词的几种写法都在、旧唤醒词「小伙伴」完全不出现。但 v2.56 相对 v2.55 只多改了时钟这一处，**尚未上板复测；语音链路（唤醒 → ASR → LLM → TTS）也没有在这一版上重新实测**——唤醒词的匹配写法只做过离线单测，没有对着真实 ASR 输出校准过。开发过程中的各功能版本（v2.53 及以前）都是在真机上验证过的。
 5. 服务端（LLM / ASR / TTS / MQTT broker）需要自行搭建，作品本身不含服务端。
-

七、目录说明

app/kid_buddy/	作品主体：1900+ 行 C + 5 个技能文件
board/nsh_minidisplay/	板级配置（defconfig + Make.defs），自带即可编译
image/	预编译整机固件 + SHA256
logs/	AI Coding 对话日志（2026-07-25 ~ 09-12）
.claude/skills/	本队沉淀的 4 个开发 Skill
docs/	作品介绍、演示脚本、上游 PR 备忘
contest2026_086_ditiekouyegoulianmeng.xml	
	manifest，把 app/kid_buddy 软链进编译树